```
#include <iostream>
#include <string>
#include <map>
#include <iomanip>
#include <stdexcept>
using namespace std;

enum class Zone {
   Inner = 1,
   Middle = 2,
   Outer = 3
};

// Base class
class Passenger {
protected:
   string passengerId;

public:
   Passenger(string id) : passengerId(id) {}

   virtual int concessionPercent() const {
      return 0;
   }

   virtual string getType() const {
      return "Regular";
   }

   virtual ~Passenger() {}
};
```

```
// Derived class
class Student : public Passenger {
public:
  Student(string id) : Passenger(id) {}

  int concessionPercent() const override {
    return 50;
  }

  string getType() const override {
    return "Student";
  }
};

// Derived class
class SeniorCitizen : public Passenger {
public:
  SeniorCitizen(string id) : Passenger(id) {}

  int concessionPercent() const override {
    return 50;
  }

  string getType() const override {
    return "Senior Citizen";
  }
};

// Smart card class
class SmartCard {
```

```
private:
    string cardId;
    long long balancePaise;
    static const long long MIN_BALANCE = 5000;

public:
    SmartCard(string id, long long balance)
        : cardId(id), balancePaise(balance) {
        if (balance < 0)
            throw invalid_argument("Invalid balance");
    }

    string getCardId() const {
        return cardId;
    }

    long long getBalance() const {
        return balancePaise;
    }

    void recharge(long long amount) {
        if (amount <= 0)
            throw invalid_argument("Recharge must be positive");

        balancePaise += amount;
    }

    bool canPay(long long fare) const {
        return fare >= 0 &&
               balancePaise - fare >= MIN_BALANCE;
    }
```

```
    bool deductFare(long long fare) {
        if (!canPay(fare))
            return false;

        balancePaise -= fare;
        return true;
    }
};

// Fare table class
class FareTable {
private:
    map<Zone, long long> fares;

public:
    FareTable() {
        fares[Zone::Inner] = 1000;
        fares[Zone::Middle] = 2000;
        fares[Zone::Outer] = 3000;
    }

    long long getFare(Zone zone) const {
        auto it = fares.find(zone);

        if (it == fares.end())
            throw invalid_argument("Invalid zone");

        return it->second;
    }
};
```

```
// Fare calculator class
class FareCalculator {
private:
    const FareTable& fareTable;

    bool isPeakHour(int hour, int minute) const {
        if (hour < 0 || hour > 23 ||
            minute < 0 || minute > 59) {
            throw invalid_argument("Invalid travel time");
        }

        int time = hour * 60 + minute;

        bool morningPeak = time >= 480 && time < 600;
        bool eveningPeak = time >= 1020 && time < 1200;

        return morningPeak || eveningPeak;
    }

public:
    FareCalculator(const FareTable& table)
        : fareTable(table) {}

    long long calculateFare(
        Zone zone,
        const Passenger& passenger,
        int hour,
        int minute
    ) const {
        long long fare = fareTable.getFare(zone);
```

```
        if (isPeakHour(hour, minute))
            fare = fare * 120 / 100;

        int concession = passenger.concessionPercent();
        fare = fare * (100 - concession) / 100;

        return fare;
    }
};

// Journey class
class Journey {
private:
    string journeyId;
    Zone zone;
    int hour;
    int minute;

public:
    Journey(string id, Zone z, int h, int m)
        : journeyId(id), zone(z), hour(h), minute(m) {}

    bool completeJourney(
        SmartCard& card,
        const Passenger& passenger,
        const FareCalculator& calculator
    ) const {
        long long fare = calculator.calculateFare(
            zone, passenger, hour, minute
        );
```

```
        cout << "\nJourney ID: " << journeyId << '\n';
        cout << "Passenger Type: "
             << passenger.getType() << '\n';
        cout << "Fare: Rs. " << fixed << setprecision(2)
             << fare / 100.0 << '\n';

        if (!card.deductFare(fare)) {
            cout << "Journey rejected: minimum balance "
                 << "of Rs. 50 must remain.\n";
            return false;
        }

        cout << "Journey successful.\n";
        cout << "Remaining Balance: Rs. "
             << card.getBalance() / 100.0 << '\n';

        return true;
    }
};

int main() {
    try {
        FareTable fareTable;
        FareCalculator calculator(fareTable);

        SmartCard card("CARD101", 10000);
        Student passenger("STU101");

        Journey journey("J001", Zone::Middle, 8, 30);
```

```
        cout << "NAMMA METRO SMART-CARD FARE SYSTEM\n";
        cout << "Card ID: " << card.getCardId() << '\n';
        cout << "Initial Balance: Rs. "
             << card.getBalance() / 100.0 << '\n';

        journey.completeJourney(card, passenger, calculator);
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```